Appello del 9 Febbraio 2018



Politecnico di Milano

Anno accademico 2017-2018

Ingegneria del Software

Cognome:		LAUREANDO ☐
Nome:	Matricola:	
Sezione (segnarne una): ☐ Cugola	☐ Mottola	☐ San Pietro

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare solo i fogli distribuiti utilizzando il retro delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. non verranno in nessun caso presi in considerazione. È possibile scrivere in matita.
3. È possibile consultare liberamente libri, manuali o appunti. È proibito l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, smartwatch, ...).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. La consegna di un compito gravemente insufficiente comporta l'impossibilità a partecipare all'appello successivo.
6. Tempo a disposizione: 2h.

Esercizio 1:

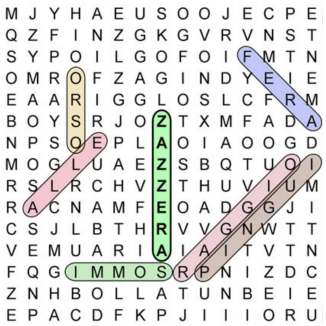
Esercizio 2:

Esercizio 3:

Esercizio 4:

Esercizio 1

Si consideri un programma Java, rivolto ad appassionati di enigmistica, per la soluzione di semplici Cruci-puzzle, di cui viene dato un esempio in figura.



Il gioco consiste nel selezionare parole di senso compiuto all'interno della scacchiera: dall'alto al basso, dal basso all'alto, da sinistra a destra o da destra a sinistra, ma anche in diagonale. Ci sono quindi 8 direzioni possibili della selezione, che si possono individuare con i punti cardinali: N, S, E, O, NO, NE, SO, SE. A tale scopo, si definisce una classe mutabile `Selezione`, i cui oggetti contengono una sequenza di celle selezionate unitamente alla direzione della selezione (a partire dalla prima lettera, detta origine).

Si definisce poi una classe `Crucipuzzle` i cui oggetti includono il contenuto del puzzle (scacchiera) e le selezioni corrette effettuate fino a quel momento. I dettagli si possono evincere dal codice qui sotto riportato, che include solo una specifica delle varie classi.

```
public class Selezione {
  /* Costruisce una selezione a partire dalle coordinate della prima cella,
   * seguendo una direzione fissata*/
  public Selezione(int x, int y, String direzione) ;

  /* Aggiunge una cella in più alla selezione, nella direzione
   * predefinita, lanciando eccezione se si è raggiunto il bordo
  public void aggiungiCella() throws OutOfBoundsException;

  /* Ritorna indietro, deselezionando una cella;
   * lancia eccezione se si cerca di rimuovere l'origine */
  public void rimuoviCella()  throws OriginException ;

  /* Restituisce la stringa corrispondente alle celle selezionate */
  public /*@ pure @*/ String selezione() ;

  /* Restituisce la direzione della selezione */
  public /*@ pure @*/ String direzione() ;
....
}

public class Crucipuzzle {
  /* Costruisce un crucipuzzle a partire da una matrice rettangolare di
   * caratteri e da un array che contiene le parole da trovare.
  public Crucipuzzle(char[][] schema, String[] paroleContenute);

  /* Restituisce il contenuto del puzzle in una matrice rettangolare */
  public /*@ pure @*/ char[][] scacchiera();

  /* Restituisce un ArrayList con le parole ancora da individuare */
  public /*@ pure @*/ ArrayList<String> parole();
```

```
  /* Aggiunge una nuova selezione al puzzle rimuovendo la parola corrispondente
   * dall'elenco delle parole ancora da individuare; restituisce true sse la selezione
   * completa il puzzle; lancia eccezione se la selezione è relativa ad una parola
   * già trovata o che non è prevista fra la parole contenute nel puzzle. */
  public boolean aggiungiSelezione(Selezione s) throws SelezioneErrataException ;
....
}
```

Domanda a

Si specifichi in JML il metodo `aggiungiSelezione`

Soluzione

```
//@ensures \old(parole().contains(s.selezione())) &&
//@        parole().size()==\old(parole().size()-1) &&
//@        (\forall String x; \old(parole().contains(x);
//@                    x!=s.selezione() ==> parole().contains(x)) &&
//@        \result <==> (parole().size()==0);
//@signals(SelezioneErrataException e)  !\old(parole().contains(s.selezione())) &&
/@        parole().size()==\old(parole().size()) &&
//@        (\forall String x; \old(parole().contains(x); parole().contains(x));
public boolean aggiungiSelezione(Selezione s) throws SelezioneErrataException ;
```

Domanda b

E' data la seguente implementazione parziale di `Crucipuzzle`. L'array di booleani `trovata` contiene `true` alla posizione i se e solo se la parola nella stessa posizione dell'ArrayList `parole` è già stata individuata.

```
import java.util.ArrayList;
public class Crucipuzzle {
//rep:
   private char[][] celle;
   private ArrayList<String> parole;
   private boolean[] trovata;
   private ArrayList<Selezione> selezionate;
   public Crucipuzzle(char[][] schema, String[] paroleContenute) {
     selezionate = new ArrayList<Selezione>();
      ...inizializza  celle con lo schema, e parole con paroleContenute....
   }

   public /*@ pure @*/ char[][] scacchiera() {
      .......
   }


   public boolean aggiungiSelezione(Selezione s) throws SelezioneErrataException {
      .......
   }
}
```

Si scriva l'invariante di rappresentazione e si implementino i metodi `scacchiera()` e `aggiungiSelezione`.

Soluzione

```
private invariant parole!=null && celle!=null && celle[0]!= null &&
//celle e' una matrice rettangolare:
(\forall int i; 0<i && i<celle.length; celle[i]!=null &&
                                       celle[i].length==celle[i-1].length) &&
//trovata e parole hanno la stessa lunghezza:
trovata.length==parole.length &&
//l'array selezionate puo' contenere solo stringhe dell'Ar.List parole:
(\forall Selezione s ;selezionate.contains(s);
  parole.contains(s.selezione())) &&
// le parole selezionate corrispondono a quelle trovate:
(\forall int i;0 <= i && i<trovate.length; trovata[i] <==> selezionate.contains(parole.get(i))
```

```
  public boolean aggiungiSelezione(Selezione s) throws SelezioneErrataException
    String word = s.selezione();
    int pos = parole.indexOf(word);
    if (pos == -1) throw new  SelezioneErrataException();
    selezionate.add(s);
    trovata[pos]=true;
    for (int i=0; i<trovata.length; i++) {
      if (!trovata[i]) return false;
    }
    return true;
  }

  public char[][] scacchiera() {
    char[][] s = new char[celle.length][celle[0].length];
    for(int i=0; i<celle.length; i++) {
      for(int j=0; j<celle[0].length; j++) {
        s[i][j] = celle[i][j];
```

```
      }
    }
    return s;
  }
```

Si osservi che scacchiera() non puo' restituire celle (ma nemmeno un suo clone), in quanto celle e' una parte *mutabile* del rep.

Esercizio 2

Si consideri la seguente interfaccia che realizza il tipo `Mutex`. Un solo thread per volta può acquisire il `Mutex`. L'acquisizione è realizata attraverso il metodo `lock` che sospende il chiamante se il `Mutex` è già acquisito ("locked"). Il metodo `unlock` rilascia il `Mutex` per altri thread.

```
public interface Mutex {
    /** Acquisisce il mutex, sospendendo il chiamante in attesa che diventi disponibile */
    public void lock();

    /** Rilascia il mutex */
    public void unlock();
}
```

Domanda a

Si fornisca il codice della classe `MyMutex` che implementa l'interfaccia sopra riportata. Si usino solo i meccanismi base della sincronizzazione di Java senza sfruttare classi edi pacchetti `java.util.concurrent.XXX`.

Soluzione

```
public class MyMutex implements Mutex {
    private boolean locked;

    public MyMutex() { locked = false; }

    public synchronized void lock() {
        while(locked) {
            try { wait(); }
            catch(InterruptedException ex) { ex.printStackTrace(); }
        }
        locked = true;
    }

    public synchronized void unlock() {
        notifyAll();
        locked = false;
    }
}
```

Domanda b

Si consideri la seguente classe che usa un `Mutex` per evitare conflitti nell'accesso da parte di thread diversi:

```
class Foo {
    private double x, y;
    private Mutex m;

    public Foo(double x, double y) {
        m = new MyMutex();
        this.x = x; this.y = y;
    }

    public double getX() {
        double temp;
        m.lock();
        temp = x;
        m.unlock();
        return temp;
    }

    public void setX(double x) {
        m.lock();
        this.x = x;
        m.unlock();
    }

    public double getY() {
        return y;
    }
}
```

La sincronizzazione è corretta? Perchè?

Soluzione

La soluzione è corretta. L'attributo `x` viene sia letto che scritto (dai metodi `getX` e `setX`, rispettivamente), deve quindi essere "protetto" usando il mutex (l'assegnamento di un `double` è di per se una operazione la cui atomicità non è garantita in tutte le JVM). Viceversa, l'accesso all'attributo `y` e' in sola lettura, senza rischio quindi di conflitti (e' scritto solo dal costruttore).

Esercizio 3

Si considerino le seguenti dichiarazioni di classi Java.

```
public abstract class Shape {
  public abstract String getColor();
}

public class Circle extends Shape {
  public Circle(double d, String string, boolean b) {}
  public String getArea() { return "getArea-Circle"; }
  public String getColor() { return "getColor-Circle"; }
}

public class Rectangle extends Shape {
  public Rectangle(double d, double e, String string, boolean b) {}
  public String getColor() { return "getColor-Rectangle"; }
  public String toString() { return "This is a shape with four sides!"; }
}

public class Square extends Rectangle {
  public Square(double d) {
    super(d, d, null, false);
  }
  public String getArea() { return "getArea-Square"; }
  public String getSide() { return "getSide-Square"; }
}
```

Si consideri poi il seguente codice Java che utilizza le classi sopra.

```
public class Main {
   public static void main(String[] args) {

1.        Shape s1 = new Circle(5.5, "RED", false);
2.        System.out.println(s1);
3.        System.out.println(s1.getArea());

4.        Circle c1 = (Circle) s1;
5.        System.out.println(c1);
6.        System.out.println(c1.getArea());
7.        System.out.println(c1.getColor());

8.        Shape s2 = new Shape();
9.        Shape s3 = new Rectangle(1.0, 2.0, "RED", false);
10.       Rectangle r1 = (Rectangle) s3;
11.       System.out.println(r1);
12.       System.out.println(r1.getArea());

13.       Shape s4 = new Square(6.6);
14.       System.out.println(s4);
15.       System.out.println(s4.getArea());
16.       System.out.println(s4.getColor());

17.       Rectangle r2 = (Rectangle)s4;
18.       System.out.println(r2);
19.       System.out.println(r2.getArea());
20.       System.out.println(r2.getSide());
    }
}
```

Domanda a)

Si indichino dapprima quali righe del metodo `main` generano un errore di compilazione, e perchè.

Soluzione

Le righe 3, 12, 15, 19, e 20 generano un errore di compilazione perchè il metodo che viene invocato non è definito nel tipo statico della variabile. La riga 8 genera un ulteriore errore perchè non è possibile instanziare un oggetto da una classe astratta.

Domanda b)

Supponendo che tutte le righe che provocano errori in fase di compilazione siano state rimosse, illustrare l'output del programma. Se non fosse disponibile il codice del metodo eseguito a run-time, ci si può limitare a spiegare a quale classe appartiene il metodo che viene eseguito.

Soluzione

Un esempio di output è il seguente (fra parentesi il numero di riga):

```
(2) Circle@33909752
(5) Circle@33909752
(6) getArea-Circle
(7) getColor-Circle
(11) This is a shape with four sides!
(14) This is a shape with four sides!
(16) getColor-Rectangle
(18) This is a shape with four sides!
```

Notare come le prime due righe sono il risultato dell'esecuzione del metodo `toString` della classe `Object`.

Esercizio 4

Si consideri il progetto di una interfaccia grafica. I singoli elementi grafici sono denominati “widget” e rappresentano diverse modalità di interazione con l’interfaccia. I widget possono, ad esempio, essere bottoni, menù, e checkbox. Un widget può godere di diverse qualità, come ad esempio essere un widget utilizzabile per il risparmio energetico oppure un widget ad alta definizione. Ciascuna di queste qualità possiede essa stessa delle funzionalità, e può essere associata ad un widget in maniera dinamica (a run-time), se e quando opportuno. Ad esempio, la qualità “risparmio energetico” incorpora gli algoritmi per minimizzare il consumo di energia durante la visualizzazione.

Domanda a)

Disegnare un class diagram che rappresenti un possibile progetto per il sistema descritto sopra. Dire quali pattern è opportuno applicare e perchè. Non è necessario dettagliare attributi e metodi di ciascuna classe, è sufficiente rappresentare le classi utilizzando nomi sufficientemente descrittivi e le loro relazioni.

Soluzione

La soluzione più naturale è applicare un pattern decorator, dove le classi che rappresentano le singole qualità sono i “decorator”.

Domanda b)

Tutti i widget istanziati durante una specifica esecuzione devono poter notificare le azioni svolte dall'utente su di essi (ad es., il click su un bottone) ad un insieme di classi fornite dall'utente che implementano la logica applicativa. Tale insieme di classi non è noto a priori e può cambiare dinamicamente.

Dire quali pattern è opportuno applicare in questo caso e perchè. Disegnare un sequence diagram che rappresenta una possibile interazione tra un widget e tale insieme di classi fornite dall'utente, secondo il/i pattern prescelto/i. E' sufficiente indicare classi e oggetti coinvolti, e dare un nome sufficientemente descrittivo ai messaggi che questi si scambiano.

Soluzione

La soluzione più naturale è applicare un pattern observer. Le classi fornite dall'utente si registrano ad un "manager" che viene informato dai widget dei loro cambi di stato e notifica le classi utente di conseguenza.